

Para os grafos, foi utilizado o mesmo da A1 para o não-dirigido e ponderado, e um adaptado para dirigidos e não-ponderados, com a adição de uma função para achar a transposta, utilizada no exercício 1.

Exercício 1 - CFC's: O algoritmo utilizado foi baseado nas anotações da disciplina.

- vetores C, T, F e A: Vetores conhecido, tempo descoberta, tempo finalização e antecessor, respectivamente (também as X_GT, que são os vetores para os grafos transpostos).
 - Uso de n+1 para permitir a indexação direta, posição 0 não é utilizada (definição mantida nos outros Exercícios).
- std::numeric_limits<double>::infinity: Função de limits que retorna o valor infinito positivo, usado para marcar se é inacessível (vai ser usado em outros Exercícios).
- std::vector<int> ordemVertices: Vetor auxiliar para ordenar os vértices pelo sort.
- std::vector<int> componente: Vetor que acumula temporariamente os vértices de uma mesma CFC durante a DFSVisit do grafo transposto, e permite a impressão separada por vírgula antes de passar para a próxima componente.
- std::vector<int> vizinhos: Vetor para armazenar os vizinhos do vértice sendo explorado em DFSVisit (vai ser usado em outros Exercícios).

Exercício 2 - Ordenação Topológica: O algoritmo utilizado foi baseado nas anotações da disciplina.

- vetores C, T, F: Vetores conhecido, tempo descoberta e tempo finalização, respectivamente.
- std::vector<int> O: Vetor que vai armazenar a sequência final da ordenação.
- std::vector<int> vizinhos: Mesma coisa das CFC's, usado em DFSVisit_OT.

Exercício 3 - Kruskal: O algoritmo utilizado foi baseado nas anotações da disciplina.

- vetores A_u e A_v: Vetores de inteiros que armazenam os vértices de origem e destino usados para a AGM.
- std::vector<std::vector<int>>> S: Vetor que funciona como uma de lista de listas. Cada índice é um vértice e guarda a lista de todos os vértices que atualmente pertencem a mesma componente conectada que ele.
- vetores E_u e E_v: Vetores de inteiros que armazenam os vértices de origem e de destino de cada aresta de forma linear. A condição $u < v$ garante que cada aresta só vai ser registrada uma vez.
- std::vector<int> vizinhos: Mesma coisa das CFC's.
- std::vector<int> ordemVertices: Mesma coisa das CFC's.
- std::vector<int> x: Vetor para a fusão de S[u] e S[v], usado para atualizar a referência de todos os vértices na nova componente.